BELGIUM CAMPUS ITVERSITY

SEN381

BUGBUSTERS

CIVICCONNECT PROJECT TEAM

Assignment 2

Research for Engineering Decisions

TEAM MEMBERS

CHRISTIAN JANSE VAN RENSBURG (601840)
MARCO REINERS (578056)
WALDO BLOM (578068)

PROJECT DOCUMENT DETAILS

COURSE	Software Engineering 381 (SEN381)
PROJECT	Assignment 2
SUBMISSION DATE	[15 September 2026]

Table of Contents

Design Quality and Design Patterns (Task 1) 3
The Design Problem 3
Coupling and Cohesion 3
SOLID Principles 3
CivicConnect Design Problems 4
Research and comparison of candidate patterns/approaches 4
Design Problem 1 4
Design Problem 2 7
Evidence-based CivicConnect recommendations 9
Design Problem 1 9
Design Problem 2 9
Researching Persistence and Data-Integrity Decisions (Task 2) 9
Chosen Business Operation 9
Engineering Choices 9
Transactions and Atomicity 9
Validation and Business Rules Allocation 10
Concurrency, Consistency, and Integrity 10
Performance Techniques and the Caching trade-off 10
Recommendation feeding the project 11
Researching APIs and Integration Decisions (Task 3) 12
Identify an integration problem 12
Compare integration approaches 13
An additional approach to consider (SMTP) 15
Recommendation feeding the project 15
Researching Collaborative Engineering and CI Controls (Task 4) 16
SCM and collaborative integration 16
Version Control vs. Software Configuration Management 16
Branching, Pull Request, Peer-Review and Traceability Practices 16
Integration Risk and Control 17
Why this automated CI supports but does not replace our human review process 18
Preliminary choice: GitHub Actions and Workflows 18
Research-to-Decision Map (Task 5) 20
AI usage register 21
References 22

Design Quality and Design Patterns (Task 1)

The Design Problem

Coupling and Cohesion

In software engineering coupling and cohesion describes how the parts of a system relate to each other.

- Coupling refers to the level of dependency between software classes or components. Classes are considered coupled when functionality in one class relies on methods or data defined in another, and higher coupling increases the likelihood that a change to one class will affect other classes in the system (Sommerville, 2016). Therefore, when software systems are designed the aim is to design systems where the components are loosely coupled (components are relatively independent) thereby reducing the ripple effect of change.
  This ensures that:
  1. Changing one component is less likely to break others
  2. Maintenance is easier because changes stay more localized
  3. Testing is simpler because components can be tested independently
  4. Components are easier to replace or upgrade
  5. Bugs are easier to isolate
  6. Multiple developers can work on different components with fewer conflicts.

  It is important to note that there can't be zero coupling, because useful components still need to collaborate.

- Cohesion describes how closely the responsibilities within a component belong together. High cohesion means that a component remains focused on a clear and related set of responsibilities (Zafar, 2026). In short: Higher cohesion makes a component better organised, easier to understand, simpler to test and maintain, and more reusable.

These concepts become particularly important when a change in the overall system requires several classes or components to be modified together. High coupling can cause changes to propagate through the system and increase the amount of code that must be retested, whereas high cohesion and lower coupling help keep changes more isolated and allow components to be maintained and tested more independently (Sommerville, 2016; Zafar, 2026)

SOLID Principles

SOLID is an acronym representing five essential software engineering principles:

1. **S**ingle Responsibility Principle (SRP): Helps Improve cohesion.
2. **O**pen/Closed Principle (OCP): Helps manage expected variation.
3. **L**iskov Substitution Principle (LSP): Supports reliable polymorphism.
4. **I**nterface Segregation Principle (ISP/IPS): Reduces unnecessary dependencies.
5. **D**ependency Inversion Principle (DIP): Supports replaceability and testability.

The SOLID principles provide guidance for structuring object-oriented software in a way that limits unnecessary dependencies and supports maintainability and future change. The principles are designed to combat software issues such as rigidity, where changes break unrelated parts; fragility, where bugs appear in unexpected places; and immobility, where code cannot be reused (Weisfeld, 2019) .

For Civic Connect, the Single Responsibility Principle (SRP), Open/Closed Principle (OCP), and Dependency Inversion Principle (DIP) are particularly relevant to the design problems anticipated before substantial implementation.

- **Single Responsibility Principle (SRP)**: A module or class should have one reason to change, meaning that it should remain focused on a clearly defined responsibility (Martin, 2000). This is relevant to CivicConnect because request processing, notifications, auditing and reporting represent different responsibilities. Combining these concerns within one component would give that component several unrelated reasons to change and could reduce its cohesion.
- **Open/Closed Principle (OCP)**: Software entities should be open for extension but closed for modification, meaning that new behaviour should ideally be introduced without repeatedly changing stable existing logic (Martin, 2000). This is relevant to CivicConnect because additional actions may need to occur as the request lifecycle evolves. The design should therefore avoid requiring substantial modification of core request-processing logic whenever new behaviour is introduced.
- **Dependency Inversion Principle (DIP)**: High-level software components should depend on abstractions rather than directly on concrete lower-level implementations (Martin, 2000). This is relevant where CivicConnect's core business logic interacts with supporting functionality such as notifications or other services. Depending on abstractions can reduce direct coupling between components and make implementations easier to replace or test independently.

CivicConnect Design Problems

Design Problem 1: Multiple responsibilities may become directly tied to request status changes

A request status change in CivicConnect affects more than only the stored status of the request. The current requirements state that staff must update requests through controlled status transitions and that requesters must receive feedback when their requests are accepted, rejected, updated or completed. The system must also maintain request lifecycle information for auditing and provide management with reporting information about request activity (Blom *et al*., 2026) This creates a design problem because the component responsible for the request workflow could become directly dependent on several separate responsibilities, including notifications, auditing and reporting. If these responsibilities are implemented together, changes to one concern could require changes to the request-processing logic even where the underlying workflow has not changed. This would increase coupling and reduce cohesion before the system has been substantially implemented.

Design Problem 2: Request workflow rules may become duplicated across multiple application components

CivicConnect contains several business rules that determine which users may perform actions and how requests may progress through their lifecycle. Staff may accept responsibility for requests, update their status and resolve or close them, while management may assign or reassign requests and exercise additional control over request handling (Blom *et al*., 2026). Because request status changes must follow controlled transitions, a design problem would arise if these rules were implemented separately across controllers, services or user-interface components. The same transition or authorisation rule could then exist in several locations, creating the risk of inconsistent behaviour and requiring multiple classes to change whenever the workflow is updated. This problem should therefore be addressed before substantial implementation so that the workflow rules remain consistent and easier to maintain.

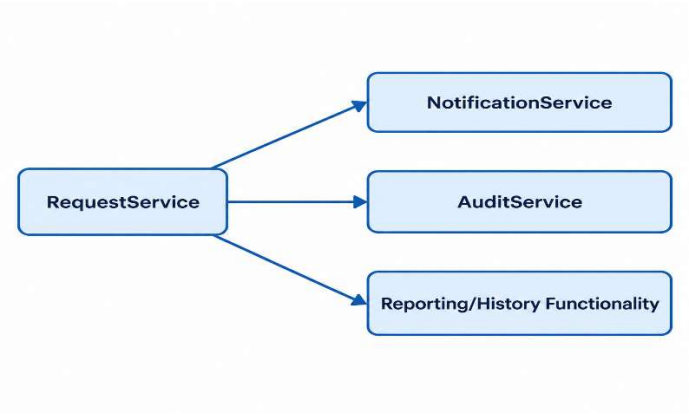
Research and comparison of candidate patterns/approaches

Design Problem 1: Multiple responsibilities may become directly tied to request status changes

Candidate 1: Direct Service Orchestration

The intent of direct service orchestration is to keep the entire process under the control of one central application service. The service responsible for changing a request status would perform the status update and then directly call the other services that are required, such as notification or audit services.

Example of how this might look like:



The responsibility of the main request service is therefore not only to control the request workflow, but also to coordinate what happens after the workflow changes.

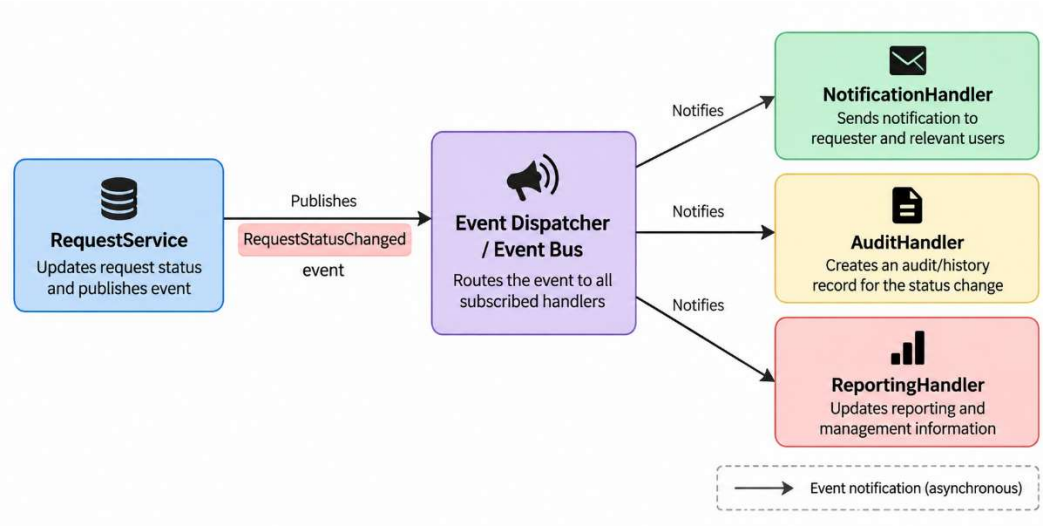
- Coupling: This approach can create higher coupling because the request service needs direct knowledge of each supporting component it calls.
- Cohesion: Cohesion may decrease if the request service becomes responsible for coordinating several concerns that do not directly belong to request workflow logic.
- Maintainability: It is easy to maintain while the number of dependent actions is small but becomes harder as more functionality is added.
- Extensibility: Adding another reaction to a status change usually requires modifying the request service itself.
- Testability: Testing initially is plain and simple, but more dependencies may need to be mocked as the service grows, making it harder to test.
- Complexity: The main advantage is its simplicity. The execution flow is easy to follow because everything happens explicitly from one place.
- Over-engineering risk: There is little risk of over-engineering, but there is a risk that the central service grows into a large service that knows too much about unrelated parts of the application.

This approach is suitable when only a small and stable number of actions must occur after a request status changes. If CivicConnect only needed to save the new status and perhaps send one notification, direct orchestration could be the simpler and more appropriate option.

Candidate 2: Observer / Event-Based Approach

The purpose of an Observer-style approach is to separate the component that changes the request status from the components that need to react to that change. Instead of the request service directly calling every related component, it publishes or raises an event such as RequestStatusChanged. Other components subscribe to that event and perform their own responsibility when the event occurs.

Example of how this might look like in practice:



The request workflow component is responsible only for changing the request and announcing that the change occurred. Each subscribed handler is responsible for its own reaction to the event.

- Coupling: Direct coupling is reduced because the Request Service does not need detailed knowledge of every component reacting to the change.
- Cohesion: Cohesion is improved because notification, auditing and reporting logic remain in their own focused components.
- Maintainability: Improved: A change to notification logic can normally be made without modifying the core request workflow.
- Extensibility: New reactions (more handlers) can be added by introducing another subscriber/observer rather than changing the original request-status logic.
- Testability: Individual handlers can be tested separately from the request workflow.
- Complexity: Additional event classes, interfaces or handlers must be introduced, making the design more complex than direct method calls.
- Over-engineering risk: Using a full distributed event system or external message broker would likely be unnecessary for a project of CivicConnect's size. Too many events can also make the execution flow harder to follow and debug.

An Observer-style or event-based approach is more appropriate when several independent components need to react to the same change and those responsibilities may evolve separately. If there are only one or two simple reactions, direct orchestration would normally be preferable because it is easier to understand and introduces less overhead.

Comparison

Direct orchestration offers the simplest implementation and keeps the flow of execution clear, but it can cause the request service to become increasingly coupled to other responsibilities as CivicConnect grows. The Observer approach introduces additional structure but provides stronger separation between request workflow, notifications, auditing and other reactions. Therefore, direct orchestration is more suitable for a small and stable workflow, while an Observer-style approach becomes more useful where multiple independent reactions to status changes need to be supported.

Design Problem 2: Request workflow rules may become duplicated across multiple application components

Candidate 1: Centralised Workflow / Transition Service

The purpose of a centralised workflow service is to place the request-transition rules in one dedicated component. Controllers or other parts of the application would not decide for themselves whether a transition is valid. Instead, they would pass the request and intended transition to the workflow service, which would determine whether the action is permitted.

A simplified workflow could be represented as:



The workflow service would be responsible for validating these transitions and applying the relevant workflow rules.

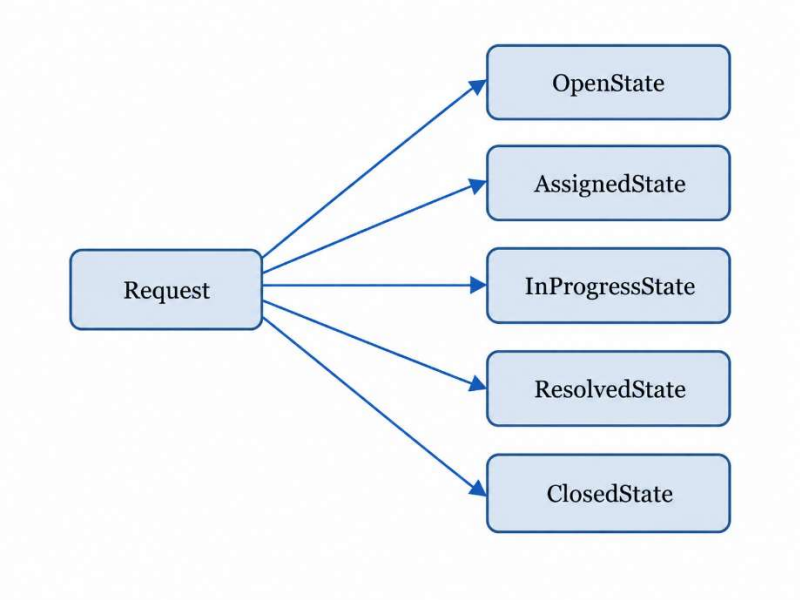
- Coupling: Controllers and other components become dependent on one workflow service rather than each implementing the rules independently.
- Cohesion: Workflow logic remains grouped in one component with a clear responsibility.
- Maintainability: A workflow change can usually be made in one place instead of updating several controllers or services.
- Extensibility: The approach works well for a small number of states and transitions, although it may become more difficult to maintain if the workflow becomes very complex.
- Testability: Valid and invalid transitions can be tested directly against one central component.
- Complexity: The design remains relatively simple because it does not require a separate class for every possible state.
- Over-engineering risk: There is little over-engineering risk initially, but the workflow service can become too large if it develops many if/else or switch statements and begins handling unrelated behaviour.

This approach is suitable when the main concern is ensuring that request transitions are valid and consistently enforced. It is especially appropriate where the number of states is limited and the different states do not contain substantially different behaviour.

Candidate 2: State Design Pattern

The intent of the State Pattern is to represent the different states of an object as separate classes or objects. Instead of one central component repeatedly checking the current request status, the request delegates state-specific behaviour to the object representing its current state.

Example of how this might look like in practice:



Each state would be responsible for defining which actions or transitions are allowed while a request is in that specific state.

- Coupling: The main request class is less dependent on large state-based conditional structures, although it now depends on a state abstraction.
- Cohesion: State-specific rules are kept inside the state to which they belong, giving each state class a focused responsibility.
- Maintainability: The design can be easier to maintain when each request state contains substantial and different behaviour.
- Extensibility: New states or state-specific behaviour can often be introduced without rewriting a large central transition method.
- Testability: Each state and its permitted behaviour can be tested independently.
- Complexity: The main disadvantage is the increase in the number of classes and interactions in the system.
- Over-engineering risk: The State Pattern can easily become unnecessary complexity if the different request states only represent simple values and have very little unique behaviour.

The State Pattern is appropriate when the behaviour of a request changes substantially depending on its current state. If CivicConnect only needs to validate a relatively simple set of permitted transitions, a centralised workflow service would usually be a simpler and more suitable solution.

Comparison

A centralised workflow service provides a straightforward way of keeping request-transition rules consistent without adding many new classes. It is easier to understand and may be more suitable for CivicConnect if the workflow remains relatively simple. The State Pattern provides stronger separation of state-specific behaviour and can improve extensibility when individual states contain significant logic. However, if the states mainly represent simple workflow stages, using the State Pattern could introduce unnecessary complexity. The simpler transition-service approach is therefore preferable unless CivicConnect develops substantially different behaviour for each request state.

Evidence-based CivicConnect recommendations

Design Problem 1: Multiple responsibilities may become directly tied to request status changes

It depends on how many separate components need to react when a request status changes, how likely those reactions are to change over time, and how much additional complexity the team is willing to introduce. If only one or two simple actions need to happen, direct service orchestration may be enough. However, if several independent responsibilities such as notifications, auditing and reporting need to react to the same status change, then a more decoupled approach becomes more suitable.

I would therefore recommend using a lightweight event-based approach such as observer for CivicConnect. It fits the project best because several separate responsibilities may need to react to the same status change, while the main request workflow can remain focused on handling the request itself.

This should reduce direct coupling between the request workflow and the other components. Separate handlers can deal with notifications, audit/history updates and reporting-related actions, which should also improve cohesion and make the system easier to maintain, extend and test.

Design Problem 2: Request workflow rules may become duplicated across multiple application components

It depends on how complex the request lifecycle becomes, how different the behaviour of each state is, and how much additional structure is actually needed. If the main requirement is simply to control which transitions are allowed and make sure the same rules are applied consistently, then a centralised workflow or transition service is likely to be enough. However, if each state later develops its own substantial behaviour and rules, then the State Pattern may become more appropriate.

I would therefore recommend using a centralised workflow service for CivicConnect. It fits the current project best because the request lifecycle is still fairly simple and the main need is to keep transition rules consistent in one place without adding unnecessary complexity.

It also avoids introducing a separate class for every request state before that level of complexity is actually needed. The State Pattern can still be reconsidered later if the behaviour of states such as Open, Assigned, In Progress, Resolved and Closed becomes significantly different or more complex.

Researching Persistence and Data-Integrity Decisions (Task 2)

Chosen Business Operation

The operation selected for this research is the service request status transition and the resolution. The context for this operation occurs when an authorised staff member updates a request status for example, “In Progress” to “Resolved” and submits a mandatory resolution comment. A Failure consequence of this operation is if the systems update the ticket status to “Resolved” but a database failure prevents the resolution comment or the timestamped log from saving, the system the becomes inconsistent in state. The management loses accountability and the core business capability of tracking the request lifecycle is corrupted.

Engineering Choices

Transactions and Atomicity

To prevent partial data writes, the status update, the comment insertion and the audit log creation must be treated as a single atomic transaction. Atomicity guarantees that either all operations within a transaction succeed or that none of them do (Kleppmann, 2017, p.223). Without atomicity, a network timeout in the middle of an operation leaves orphaned records in the database thus breaking the integrity of CivcConnect’s reporting dashboard.

Validation and Business Rules Allocation

Validation must not just reply on one architectural layer, as relying on solely one architectural layer brings security risks as well as maintainability risks.

- The UI/Client layer: Validation here for example, ensuring the comment box is not empty, is just for the user experience by giving immediate feedback without network latency. On the other hand, client-side checks can be bypassed by malicious requests, meaning that they offer zero real security and cannot enforce business invariants reliably (Richards and Ford, 2020, p.145).
- The Application layer: This is where complex business rules must be. The application layer must verify that the staff member is authorised for the department and that the transition from "Open" directly to "Resolved" is logically allowed.
- The Database layer: The database should only enforce structural integrity, such as foreign key constraints. Relying on the database for complex business logic, via stored procedures, tightly couples the application to the database vendor and makes version control exceedingly difficult (Kleppmann, 2017, p.175).

Concurrency, Consistency, and Integrity

Multiple managers or staff members might view the same ticket simultaneously, because of this there is a severe risk of the "lost update" problem. For example, staff member 1 and staff member 2 attempt to assign themselves to the same unassigned ticket at the exact same time. We compared two concurrency control approaches:

1. Pessimistic Locking. This happens on the database level. The database locks the row the moment that a user opens the ticket, that prevents other users from editing it until the transaction completes. The benefit of this is that it guarantees absolute consistency. But the consequence is that it can cause severe performance bottlenecks and poor user experience in web applications specifically. If a user closes their browser while the lock is held then the ticket will remain frozen (Kleppmann, 2017, p.258).
2. Optimistic Concurrency Control: This happens on the application level. No locks are placed when reading data. Instead, the application checks a version number or timestamp upon writing. If the record was changed by another user, then the transaction is rejected and the user is prompted to refresh the page. The benefit is that it is scalable and ideal for web applications with distributed users. But it requires handling retry logic and user-facing error messages in the application layer (Kleppmann, 2017, p.245).

Performance Techniques and the Caching trade-off

While performance techniques like in-memory caching are useable for read-heavy operations like management dashboard, they also bring a correctness vs staleness trade-off for workflows like a status transition.

- The staleness risk: If caching is introduced to the ticket process, a staff member will read a stale cached version of the ticket that appears "Open" even when another staff member just "Resolved" it in the database (Kleppmann, 2017, p. 324).
- Evidence based decision: CivicConnect requires accountability for who resolved a ticket and when, because of this, the importance of data staleness outweighs the performance that's gained of caching (Richards and Ford, 2020, p. 182). This means we will not introduce caching for the ticket resolution as well for the transition workflow. All reads and writes for this operation must query the database directly to make sure of correctness.

Recommendation feeding the project

We recommend that CivicConnect handles the status transition operation using application managed atomic transactions and paired with optimistic concurrency control. The application service layer will further enforce the business rules and wrap the database calls into a single transaction block. This research informs our M2 data persistence baseline. It requires a dedicated architecture decision record detailing our persistence framework configuration. And this decision will be tracked in the RTM under data persistence column for FR202.

Researching APIs and Integration Decisions (Task 3)

Identify an integration problem

One thing that we have already identified is that we want the user (requester) to have push notifications via email when the request changes status (from Submitted to in progress or from in progress to resolved or closed). This is required by FR-104 (feedback when a request is updated) and the in-scope feature “Feedback via email push notification” in our PED v1 for Civic Connect.

In a concurrent system processes need to synchronize their actions. When two process synchronize, it is classified as either cooperation or contention. An example of cooperation would be when there are two processes called producers and consumers; producers are the things that produce the data items which the consumers make use of (consume) (Kao, 2008, p.237).

Producer / caller:

In our case the producer would be the Request Management module which is the backend component that will be responsible for the change in status and request updates (see FR-202 and FR-206).

Consumer / callee:

The consumer would then be the actual email / notification component that we will be creating, that will send the requester the notifications. We might call an external API from an email provider. The reason being that an API provides specific functionality and capabilities, so that we as the API user don’t have to implement it ourselves from scratch (Marcin Dulak, 2025, ch. 1, para. 14).

Information exchanged during API request:

In order to make this exchange we will need the email address of the requester (we already have this as we require email for our sign up, see FR-105), but in order to get the email address we must first have the Requester ID and then use the ID field to look up the email.

Next, we need the previous status and then the new status. We will also need the timestamp of when the status change accord. We will also need the ID of the staff member in order to look up their info such as name and department. Also, for the status update if the staff member added an additional comment to the message (See FR-204) .

Example of what the message would look like:

`

Good day [requester name],

This email is with regards to [request name] with request id of [request id]. The status of the request has changed from status [prev status] to [new status]. Staff member ` [staff member name] ` had this to say:

[comment by the staff member]

Kind regards,

CivicConnect, [department name] department

`

If the notification fails:

The requester might not know if the status has updated util they sign in again, and this is not efficient. For example, for a pressing matter, the person might sit there keep on refreshing their page until they see the status updates. This is the main reasoning behind having push notifications, but if we don't implement it properly, the problem still won't be solved. We need to consider things such as:

If the notification call is synchronous and the email provider API is slow or unavailable the status update may also be delayed or even fail. If the notification retries are not implemented properly the requester might get more than one email saying status has been updated, this can be confusing or frustrating as we don't want to spam the user with emails unnecessarily.

We also need to be clear how exactly we define failure for example: what we also need to consider is that we must ensure that the email does not land up in the junk mail of the user otherwise they might miss it, and if the user could miss a message because of it routing to junk mail this could almost be as bad as the message not going through at all. Lastly, we also need to consider that the email might contain sensitive / private information so the integration must also respect the requesters and staff's privacy (see NFR-101 and FR-305), and if we violate this it could also be considered as a failure.

Compare integration approaches

Representational State Transfer (REST) is the most popular network-based API (Marcin Dulak, 2025, ch. 5, para. 1). When it comes to REST it commonly consists of three main components: HTTP, a resource and a URI. HTTP is the most widely used protocol on the web it decides the way in which the fetching of a resource actually takes place (Marcin Dulak, 2025, ch. 5, para. 3 - 12). A resource is any information that can be a target of the hypertext reference such as a document, image, numerical value etc (Marcin Dulak, 2025, ch. 5, para. 3 - 12). A resource can be static or dynamic. URI (uniform resource identifier) is used to provide an identifier to differentiate resources from one another and to identify if a resource has changed (Marcin Dulak, 2025, ch. 5, para. 3 - 12). A URI can either be a URL which includes information about an access mechanism and a location of the actual resource. Or a URN which identifies a resource without deciding how or where to access it (Marcin Dulak, 2025, ch. 5, para. 3 - 12). REST API's can also be further be classified into RESTFul or RESTless (Marcin Dulak, 2025, ch. 5, para. 49 - 51).

Async event / message has to do with what is referred to as broker-based API's (Marcin Dulak, 2025, ch. 11, para. 1 - 10). This is especially common in SOA (service-oriented architecture) which in the early 2000's allowed messages over an Enterprise Service Bus (ESB), a commination system for different applications that need to communicate with one another. It is also commonly used in EDA (starting around the mid 2000's) which allows systems to respond to each other, based on messages created by events that take place (Marcin Dulak, 2025, ch. 11, para. 1 - 10). Later on, in the 2010's microservices gained more traction and continues to use MOM (message-oriented middleware) (Marcin Dulak, 2025, ch. 11, para. 1 - 10). MOM is a software or hardware solution that facilitates sending and receiving messages among applications, though important to note this is not done directly (Marcin Dulak, 2025, ch. 11, para. 1 - 10). MOM acts as a middleman ensuring that the message are delivered from the producers to the consumer and this is also done asynchronously (Marcin Dulak, 2025, ch. 11, para. 1 - 10).

	HTTP / Rest API	Async event / message
Coupling	**Tight coupling.** They are tightly coupled to the data structures that they expose (Marcin Dulak, 2025, ch. 6, para. 4 - 6).	**Loosely component coupling.** Producers and consumers do not directly comminate with one another. A queue helps the producers and consumers of messages be more loosely coupled, this means that producers and consumers don't need to be constantly aware of each other's presence. (Marcin Dulak, 2025, ch. 11, para. 20 - 38).
Performance	**Synchronous and Blocking.** Senders must wait for a response from the server before continuing processing. Can suffer from over fetching or under fetching (Marcin Dulak, 2025, ch. 6, para. 4 - 8).	**Asynchronous and non-blocking.** Senders publish messages without waiting for processing, thereby enabling non-blocking execution (Marcin Dulak, 2025, ch. 11, para. 20 - 38).
Failure Behaviour	**Immediate Failure.** If the target server is down or slow, a request will fail immediately with the corresponding status code eg. 404, 500, etc indicating what the issue is. Or in the case that is was successful a 200 request (Marcin Dulak, 2025, ch. 4, para. 44 - 46)	**Reliable and tolerable.** Systems that use queues tend to be more reliable and tolerate to faults. For example, if a message consumer is unavailable, the message would remain in the queue until the consumer is back online and can retrieve it. Additionally, in the case of a sudden increase of messages, you could scale out the system by adding more consumers. (Marcin Dulak, 2025, ch. 11, para. 25 - 49).
Testability	**Easier Testing** Mostly easier to test and debug due to immediate, synchronous response codes. However, this depends on the implementation, if the REST API does not make use of HATEOAS it could be difficult (Marcin Dulak, 2025, ch. 6, para. 4 - 8).	**Higher Testing Complexity.** Debugging and testing are challenging because processing order and execution timing can be difficult. Message duplication is also a common problem, and could potentially slow down storage space. Also, because of the Async nature of messaging you won't know at the exact moment an error occurs (Marcin Dulak, 2025, ch. 11, para. 121 - 135).
Deployment implications	**Simpler infrastructure.** Request-response model: clients send requests directly to a service and wait for responses. HTTP is stateless per-request, making it simple. (Marcin Dulak, 2025, ch. 5, para. 33 -34).	**More difficult infrastructure (Broker Dependency.)** Requires provisioning, configuring, and maintaining dedicated message brokers (e.g., RabbitMQ, Kafka). The broker introduces operational overhead and becomes a potential single point of failure if not clustered or configured correctly. Messaging APIs are also known to be quite complex to implement (Marcin Dulak, 2025, ch. 11, para. 121-135). These APIs are also known for high costs in terms of infrastructure, testing, and deployment (Marcin Dulak, 2025, ch. 11, para. 121-135).
Security Boundary	**Exposure of endpoints.** Endpoints are exposed but REST APIs are commonly protected by industry-standard authentication and authorization such as JWT, OAuth 2.0 and also TLS (Marcin Dulak, 2025, ch. 5, para. 151).	**Decoupled Network Boundaries.** Producers and consumers do not expose direct inbound network ports to each other. Security relies on encrypted transport to/from the broker (TLS, SASL) (Marcin Dulak, 2025, ch. 11, para. 145 - 148).

Change / version compatibility	**Backward Compatibility.** In REST based API's versional and backward compatibility are difficult to achieve, and with RESTless the burden of versioning is pushed towards the developers (Marcin Dulak, 2025, ch. 5, para. 164).	**Independent / Loosely coupled** In a broker-based architecture (which messaging utilizes), clients are loosely coupled with the broker. This decoupling allows clients to be independently deployable, scalable, and maintainable. For example, clients can be deployed on different machines, in separate datacenters or cloud environments (such as AWS or Azure). They can also be written in different programming languages as well. (Marcin Dulak, 2025, ch. 11, para. 121-135).

An additional approach to consider (SMTP)

Another approach to consider is to make use of Simple Mail Transfer Protocol (SMTP), which is the industry standard protocol for sending emails directly from an application to a mail server (Pankaj, 2022). Unlike REST API calls to external providers, SMTP allows an application to establish a direct connection to an SMTP server and send emails using standard libraries that are supported in most programming languages for example Java, Python and NodeJS (Forward Email, 2026).

Implementation can therefore be easier when an SMTP service is already available, as the application generally then only needs to configure the SMTP server, port, authentication and security settings (Forward Email, 2026). However, if a pre-existing SMTP server does not exist, this infrastructure must first be established and configured (Pankaj, 2022).

Recommendation feeding the project

We have identified the need for this email push notification service. Based on the research above there are 3 potential approaches: using an already established SMTP service with built in language support, another being to utilize a HTTP / REST based external API and lastly implementing an asynchronous messaging approach.

We have decided not to make use of the asynchronous messaging approach using a message broker. While this approach could provide benefits such as loose coupling, asynchronous processing, queued delivery, tolerance of temporary failures, and the ability to scale by adding consumers. Introducing a message broker or a similar infrastructure would increase deployment, testing, and operational complexity. Additionally, messaging systems are intended for applications where message delivery is of the utmost importance and must be ensured, such as financial transactions or healthcare communications, where the loss of a single message can lead to significant economic losses or endanger lives (Marcin Dulak, 2025, ch. 11, para. 152 - 154). Email push notifications for our project does not meet this requirement.

One viable option is to make use of an already established SMTP service. For example, Forward Email (2026) states that it is possible to make use of their SMTP servers. This approach offers advantages including greater control over email delivery, lower long-term per-email costs, and reduced provider lock-in (as in the future SMTP details could just be altered while keeping our core designed functionality the same). However, further investigation into these claims and capabilities is required before committing to this direction.

Alternatively, we can use a managed REST-based email API from established providers such as SendGrid, Mailgun, or Brevo Rani (Rani Gope, 2025). This approach would shift the underlying email infrastructure, delivery management, and operational responsibility to the API provider, allowing us as the development team to integrate email functionality without managing the underlying

infrastructure. We recognized some disadvantages in this approach, including the application's dependency on the provider's API contract and version, possible rate limits, API request failures or delays, and the need for the application to handle API authentication and appropriate error handling. However, using a REST-based email API service will allow us to integrate email functionality efficiently and easily.

The choice between SMTP and REST-based API approaches will be determined in Milestone 2 based on a detailed cost analysis and scalability research. We will evaluate things such the following: pricing structure and cost per email for each approach, how costs scale with increasing email volume, which approach is more suitable long term (what approach provides the best maintainability), reliability and availability of infrastructure promised by Forward Email (2026) and the integration complexity for each option.

Given that we have not yet fully investigated the costs and long-term scalability implications of each approach, deferring this decision to Milestone 2 will allow us to make a more informed choice, instead of committing to a decision that we might later regret.

Researching Collaborative Engineering and CI Controls (Task 4)

CI/CD is a common dev ops practice which as to do with ensuring good code quality, where CI refers to Continuous integration and CD refers to continuous development (Galea, 2024). They allow development teams to automate the process of integration code changes, testing them and deploying those changes to production (Galea, 2024). This helps speed up the development cycle but also helps to maintain high-quality code (Galea, 2024).

SCM and collaborative integration

Version Control vs. Software Configuration Management

Version Control is the practice of tracking and managing changes to actual source code files over time (Atlassian, 2024c). It records things such as who made a change, when, and why (Atlassian, 2024c). It also allows reverting to previous versions and allows merging changes from multiple developers (Atlassian, 2024c). Git is an example of such a distributed version control system that allows users maintains a complete history of all commits locally and synchronizes with remote repositories (Atlassian, 2024b). It was invented by Linus Torvalds in 2005; it is one of the most widely used version control systems (though not the only one) (Atlassian, 2024b)

Software Configuration Management (SCM) on the other hand is broader and encompasses version control but also extends to managing the configuration of the entire software system throughout its lifecycle (Atlassian, 2024a). SCM includes things such as version control, build configuration, release management, change control, baseline management, audit trail, and environment configuration (Atlassian, 2024a).

For our CivicConnect project, implementing SCM means not just using Git, but also defining a workflow and change approval processes.

Branching, Pull Request, Peer-Review and Traceability Practices

Master Brief of CivicConnect requires a protected main branch and two-reviewer Pull Requests. The branch structure that we identified in Milestone 1 is as follows:

- main branch: Production-ready releases only. Protected branch that requires two approved Pull Request reviews before merge.
- dev branch: An additional integration branch that we identified in Milestone 1, where features are tested together.

- feature branches: Individual developers create feature branches from dev, work independently, then create Pull Requests back to dev.

(This branch structure is informed by established branching approaches (Driessen, 2010; Fowler, 2024)).

However, after the submission of Milestone 1, we identified a potential collaboration risk: the bottleneck created by requiring two reviewers for the dev branch in our 3-person team. If 1 of the 2 reviewers are not available this could can stall progress and lead to larger, riskier merges later.

In order to mitigate this, we are changing our workflow in order to allow the dev branch to require only 1 independent approval for merges (a reviewer who is not the same as the person who wrote the code). This balances peer review with the size of the small team, allowing for faster integration while still catching defects early. The main branch will continue to require two approvals in order to ensure production stability, as per the Master Project Brief. We will also document this change on our decision-log.md and our github-governanace.md.

Integration Risk and Control

A likely collaboration/integration risk in CivicConnect is merge conflicts and integration bottlenecks. If we were to modify shared files or if feature branches remain isolated for too long, Git merge conflicts become likely and difficult to resolve (Fowler, 2024). The original proposed two-reviewer requirement for dev could have further delayed integration.

Our plan to avoid this as much as possible is to use short-lived feature branches that are merged into dev as frequently as possible (Fowler, 2024). The refined one-reviewer policy for dev also addresses the bottleneck risk. Continuous Integration (CI) will also help identify integration problems early by automatically building and testing, ensuring that code from different developers works together before it is merged. This is discussed further in the next section.

Continuous Integration and Quality Gates

Purpose of CI and Distinction from Git/Deployment Pipelines:

Continuous Integration (CI) has to do with automatically building, testing and validating code changes as frequently as possible in order to detect integration problems early (Fowler, 2024).

CI is not the same as just having a Git repository, neither is it just having an automated deployment pipeline. CI is the automated execution of build and test suites on things such as every commit or Pull Request (Fowler, 2024;Atlassian, 2024a). This allows for fast feedback, acting as a quality gate.

Triggers and Repeatable Build Requirements

Proposed CI triggers for CivicConnect are:

- On Pull Request creation: full build suite (compile, unit tests, integration tests, static analysis, security scan).
- On merge to dev: same full suite, plus deployment to a staging environment. Where integration among different components are tested.
- On merge to main: full suite plus deployment to production.

(As suggested by (Driessen, 2010; Fowler, 2024)).

Automated Checks and Blocking Rules

Examples of recommended quality gates include: linting, type checking, unit tests, integration tests, build/compilation, dependency vulnerability scanning, security scanning (SAST), secret scanning, and code coverage (Fowler, 2024).

Checks that should block merging include linting, type checking, unit tests, integration tests, build/compilation, and secret scanning (Fowler, 2024).

These decisions will allow for good code quality however we might not include all of them, as this may depend on the tech stack that we choose. For example, linting might not be necessary when utilizing a language like Java as we will make use of an IDE that provides this for us. Also, we don't yet know how we will exactly preform these unit and integration checks, this will further be investigated in Milestone 2, but in assignment 2 we have identified the importance of it.

Secrets and Configuration Handling

Secrets (API keys, database passwords, JWT secrets) will never be committed to Git. They should be injected via CI secrets (e.g., GitHub Actions secrets). And when working with API keys and environment variables we will make use of something such as a .env file or the language related option that won't be committed to GitHub in order to ensure secrets are never leaked.

Why this automated CI supports but does not replace our human review process

Automated CI checks are effective at detecting many defects through automated builds, tests, linters, and vulnerability scanners. However, Fowler (2024) notes that automated testing is imperfect and cannot guarantee the absence of bugs. Therefore, automated CI checks should complement, rather than replace, human verification.

CI results will appear directly on the Pull Request page as status checks and allow us as developers to review the CI run (Petriella, 2025.

In other words, we as developer will still review the code must therefore check: Does this correctly implement the requirement as specified / identified? Is the code maintainable? Are error cases handled? Are tests adequate?

Preliminary choice: GitHub Actions and Workflows

With GitHub Actions we can implement a lot of different CI/CD pipelines based on a variety of different tech stacks (GitHub, 2025). Since we are already pushing our code to GitHub, making use of GitHub's built-in workflows and actions is the easiest and most logical way to implement CI/CD, this is also recommended by Petriella (2025). Additionally, since our repository is public, we are also allotted a free number of minutes and storage for our GitHub actions and workflows, these limits are generous for small to mid-size teams (Petriella, 2025).

Important to note that we will go into more depth of the exact calculations and how many actions we can run, for example if we might only go with 2 out of the 3 proposed CI triggers if we foresee in Milestone 2 that the cost will be above the free tier.

We have not yet established our tech stack (the final tech stack decision will be made in Milestone 2), but we can already decide how to utilize the different parts of GitHub Actions and Workflows in order to understand what each part does and how to adapt it based on the final tech stack decision. We are not defining the exact structure of the YAML files here. Instead, we are identifying the different ways that we can utilize this tool.

GitHub Actions is a CI/CD platform that allows us to automate our build, test, and deployment pipeline (GitHub, 2025). Workflows can be created that run on a schedule, on specific events (like a push or pull request), or manually. Actions are the individual tasks that make up a workflow, and you can use pre-built actions from the GitHub Marketplace, or we can create our own (GitHub, 2025).

GitHub Workflows are configurable automated processes that run one or more jobs (GitHub, 2025). Workflows are defined by a YAML file in the repository's .github/workflows directory. They can be

triggered by events, run on different operating systems, and execute a series of steps (actions or shell commands). For our project, we can define workflows for:

- Build and Test: Runs on every PR to dev and main in order to run for example maybe linters, unit tests, and integration tests.
- Deploy to Dev: Runs on merge on the dev branch.
- Deploy to Production: Runs on merge to main to deploy to production (main branch).

Important to note is that GitHub Actions has a lot of flexibility (GitHub, 2025). Whether our final stack is Node.js, Python, Java, or .NET, there are pre-built actions and community workflows to support our CI/CD needs (Petriella, 2025). This allows us to defer the final tech stack decision without delaying our CI/CD planning. Also important to note is that there are other CI/CD tools such as Jenkins which offer similar capabilities, but GitHub Actions provides the tightest integration with our existing repository and requires minimal infrastructure changes as this is a Cloud hosted CI/CD platform (Petriella, 2025). We realize that setting up the GitHub Actions / Workflows still requires work but there are also community established workflows that we can utilize and then modify.

Other software such as Jenkins is also free (to a certain extent) and open source, it is also widely used within the software development industry (Petriella, 2025). However, this requires a self-hosted CI/CD server which then requires additional infrastructure that needs to be implemented and maintained (Petriella, 2025).

Research-to-Decision Map (Task 5)

Assignment 2 finding	Evidence/alternative considered	Recommendation	Project decision it should inform	Expected PED/ADR/RTM/application evidence
Design problem 1: Request changes are tightly coupled to multiple separate responsibilities (notifications, auditing, reporting)	Considered direct service orchestration vs an observer / event-based approach. Direct orchestration forces the request workflow to coordinate unrelated tasks which reduces cohesion. The observer approach decouples the status trigger from the execution.	Implement an observer/ event-based approach to decouple the core request workflow from secondary responsibilities like email notifications and audit logging.	Initial design decision. Decoupling workflow responsibilities.	ADR: Notification and auditing design pattern. PED: Initial design decisions sections. Application: Implementation of publisher/ subscriber event interfaces.
Design problem 2: Request workflow transition rules and authorisations risk becoming duplicated across multiple application components.	Considered a centralised workflow / transition service vs the state pattern. The state pattern would introduce unwanted complexity, a separate class for every state, before the request lifecycle requires it.	Implement a centralised workflow service to keep transition rules consistently enforced in one place without over-engineering the solution.	Initial design decision. Workflow transitions.	ADR: Centralised workflow services. RTM: Design column updated for FR-202 workflow. Application: Centralised transition service class implementation.
Persistence / data: Simultaneous ticket assignments by multiple staff or managers create a “lost update” concurrency risk.	Considered pessimistic locking (database level) vs optimistic concurrency control (application layer) pessimistic locking creates web bottlenecks while optimistic concurrency control is highly scalable (Kleppmann, 2017).	Implement application managed atomic transactions paired with optimistic concurrency control.	Data and persistence baseline.	ADR: Persistence and concurrency strategy. PED: Data model section, showing version/ timestamp tracking columns) Application: Concurrency exception handling in the service layer.
Integration / API: The system requires email push notifications, but the 3-person team lacks the	Considered a managed HTTP/REST-based email API vs async event/ message brokers vs SMTP. Further investigation is required into SMTP	Integrate an established, managed REST-based email API to handle notifications, shifting operational infrastructure	Initial API/Integration or SMTP considered. No final decision has yet been made.	ADR: External notifications API boundary. PED: Initial integration decisions section. Application: API client implementation calling the selected REST provider.

capacity to manage underlying email infrastructure.	and HTTP, in order to make an informed decision. The decision is deferred to Milestone 2	responsibility to the provider. Or make use of an established SMTP server.		
SCM / CI: Uncontrolled code integration creates defects and requiring 2 reviewers on all branches creates bottlenecks for a 3-person team.	Considered manual code review vs automated continuous integration (CI) pipelines (Fowler, 2024). Evaluated branch protection rules balancing team size against production stability.	Implement GitHub actions CI pipelines as quality gates (build test) prior to human review. Require 1 PR approval for the dev branch to prevent bottlenecks and 2 PR approvals for the main branch.	Collaborative engineering and CI controls.	PED: Updated GitHub governance, CI rulesets and 1 reviewer policy. Application: YAML configuration files for GitHub actions CI triggers.

AI usage register

Our AI usage is documented on our GitHub repository as per the Master brief specifications.

For our AI usage register please see our GitHub repository (docs/registers/ai-usage-register.md): https://github.com/Waldo-Blom/SEN381_Bugbusters/blob/main/docs/registers/ai-usage-register.md

Or view the pull request relating to the add of the Assignment 2 AI usage: https://github.com/Waldo-Blom/SEN381_Bugbusters/pull/27

References

Atlassian (2024a). *Source code management | atlassian git tutorial*. [online] Atlassian. Available at: https://www.atlassian.com/git/tutorials/source-code-management [Accessed 14 Sept. 2026].

Atlassian (2024b). *What is git: Become A pro at git with this guide | atlassian git tutorial*. [online] Atlassian. Available at: https://www.atlassian.com/git/tutorials/what-is-git [Accessed 14 Sept. 2026].

Atlassian (2024c). *What is version control*. [online] Atlassian. Available at: https://www.atlassian.com/git/tutorials/what-is-version-control [Accessed 14 Sept. 2026].

Blom, W., Janse van Rensburg, C. and Reiners, M. (2026) *CivicConnect Project Team: Milestone 1 – Engineering Foundation & Requirements Baseline, PED v1.0*. Belgium Campus iTversity. Unpublished SEN381 project document.

Driessen, V. (2010). *A successful git branching model*. [online] nvie.com. Available at: https://nvie.com/posts/a-successful-git-branching-model/ [Accessed 14 Sept. 2026].

Forward Email (2026). *SMTP integration examples - code samples for all languages*. [online] Forward Email. Available at: https://forwardemail.net/en/guides/smtp-integration [Accessed 13 Sept. 2026].

Fowler, M. (2024). *Continuous integration*. [online] martinfowler.com. Available at: https://martinfowler.com/articles/continuousIntegration.html [Accessed 15 Sept. 2026].

Galea, D. (2024). *Top 10 Ci/CD tools for DevOps and developers*. [online] Orca Security. Available at: https://orca.security/resources/blog/top-10-ci-cd-tools-devops/ [Accessed 14 Sept. 2026].

GitHub (2025). *Understanding GitHub actions - GitHub Docs*. [online] GitHub Docs. Available at: https://docs.github.com/en/actions/get-started/understand-github-actions [Accessed 14 Sept. 2026].

Kao, M.-Y. (2008). *Encyclopedia of algorithms*. Springer Science & Business Media, p.237.

Kleppmann, M., 2017. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media.

Marcin Dulak, L.D. (2025). *Learning API styles*. [online] O'Reilly Media, Inc. Available at: https://learning.oreilly.com/library/view/learning-api-styles/9781098153984/ [Accessed 12 Sept. 2026].

Martin, R. C., 2000. *Design Principles and Design Patterns.* 1 ed. s.l.:Robert C. Martin.

Pankaj (2022). *JavaMail example - send mail in java using SMTP | DigitalOcean*. [online] www.digitalocean.com. Available at: https://www.digitalocean.com/community/tutorials/javamail-example-send-mail-in-java-smtp [Accessed 14 Sept. 2026].

Petriella, K. (2025). *Jenkins vs GitHub actions vs testkube: CI/CD testing vs test orchestration*. [online] testkubeio. Available at: https://testkube.io/blog/jenkins-vs-github-actions-vs-testkube [Accessed 14 Sept. 2026].

Rani Gope, J. (2025). *8+ Best free email API for developers in 2025*. [online] FluentSMTP. Available at: https://fluentsmtp.com/articles/free-email-api/ [Accessed 15 Sept. 2026].

Richards, M. and Ford, N., 2020. *Fundamentals of Software Architecture: An Engineering Approach*. Sebastopol, CA: O'Reilly Media.

Sommerville, I., 2016. Software engineering. 10 ed. Boston: Pearson Education Limited.

Weisfeld, M., 2019. *The Object-Oriented Thought Process.* 5 ed. Boston: Pearson Education, Inc.

Zafar, W., 2026. *Wasil Safar:Part 7: Modularity, Coupling & Cohesion.* [Online]
Available at: https://www.wasilzafar.com/pages/series/software-engineering/software-engineering-part07-modularity-coupling-cohesion.html
[Accessed 14 09 2026].